

Title: Delivery and Shipment Tracking Data Warehouse

Author: Muhammad Bilal

bilalmubeen99@gmail.com

Abstract

This project presents the design and implementation of a **Delivery and Shipment Tracking Data Warehouse** developed to streamline the management and analysis of logistics data within a structured environment. The system consolidates delivery information from multiple sources such as courier logs, customer records, and route details stored in Excel or CSV formats. Using **MySQL / SQL Server** as the primary database platform, an **ETL (Extract, Transform, Load)** process is employed to clean, transform, and integrate data into a unified schema. Key analytical capabilities include performance and delay monitoring, courier benchmarking, route and vehicle optimization, and trend analysis for forecasting shipment volumes. The project demonstrates how a well-designed data warehouse enhances decision-making by providing accurate, timely, and actionable insights derived from historical shipment data. Testing and validation confirmed data accuracy, referential integrity, and optimized query performance. The system serves as an academic prototype illustrating the practical application of data warehousing principles in the logistics and delivery domain.

Keywords

Data Analysis, Data Warehouse, Delivery Performance, ETL, Logistics, MySQL, Shipment Tracking, SQL Server, Star Schema

Table of Contents

1. Motivation 3

2. Problem Statement 3

3. Introduction 4

4. Methodology 4

 4.1. Data Collection..... 4

 4.2. Data Warehousing..... 5

 4.3. Feature Engineering 7

 4.4. System Block Diagram 8

 4.5. Model Building 8

 4.6. Deployment 9

 4.7. Testing 9

 4.8. Reporting 10

5. Results..... 11

6. Conclusion..... 19

7. References 19

1. Motivation

In today's fast-paced world, efficient logistics and on-time delivery are critical success factors for e-commerce, retail, and supply chain management businesses. Many companies manage thousands of shipments each day, including multiple carriers, delivery routes, and customer locations. However, traditional operational databases are optimized for transaction processing rather than analytical decision making. Data warehouses provide an integrated environment for analyzing supply trends, delivery performance, route efficiency, and customer satisfaction. By centralizing shipping and shipment data from multiple sources, decision makers can identify delays, optimize delivery routes, and predict shipment volumes. The goal of this project is to develop a scalable shipping and delivery tracking data warehouse that supports analytical queries and performance reporting. The system helps organizations gain actionable insights into delivery times, regional performance, carrier performance, and consumer demand patterns, enabling them to make better operational and strategic decisions.

2. Problem Statement

Logistics companies and online retailers generate vast amounts of shipping-related data every day, including shipping records, carriers, routes, vehicle information, customer reviews, and warehouse operations. This data is often spread across multiple transactional systems, such as reservation databases, tracking portals, and warehouse management software. This fragmentation creates challenges for administrators when trying to perform complex analyzes such as:

- Determining the **average delivery time** across regions.
- Identifying **delayed shipments** and their root causes.
- Comparing **courier performance** over time.
- Analyzing **shipment volume trends** by product type or destination.

Without a central repository, these queries require manual data extraction and aggregation, which is time-consuming, error-prone, and inefficient.

The proposed shipping and tracking data warehouse aim to solve these problems in the following ways:

1. Develop a hub-and-spoke design that integrates key entities such as carriers, carriers, customers, vehicles, and delivery routes into a single structure.
2. Perform ETL (extract, transform, load) operations to clean, transform, and load data from the operating system to the warehouse.
3. Support decision-making with aggregate metrics such as on-time delivery rate, average delivery time, courier efficiency, and delivery cost analysis.

This data warehouse serves as a robust analytics platform to monitor logistics performance, identify inefficiencies, and improve service quality through data-driven decision-making.

3. Introduction

The logistics and transportation industry are increasingly relying on real-time data to enable efficient operations and customer satisfaction. From booking to shipping to delivery confirmation, each shipment generates multiple data points. Managing and analyzing this information across multiple systems is a major challenge.

Data warehouses constitute effective solutions for integrating and analyzing disparate data. Unlike operational databases that focus on day-to-day transactions, data warehouses are designed for analytical queries, reporting, and historical trend analysis.

This project will develop a shipping and tracking data warehouse to integrate data from various sources such as order management systems, delivery tracking tools, and customer feedback platforms. The warehouse is modeled after using a star schema consisting of a central fact table (shipping fact) and several dimension tables (courier, customer, vehicle, route, date).

The system enables logistics managers to perform complex analytical queries. For example, analyze the total number of deliveries within a region, track delivery delays, calculate courier productivity, predict freight loading for future periods, etc. Using an ETL process, data is extracted from the source system, transformed to ensure consistency, and loaded into the warehouse for analysis.

Ultimately, this project aims to demonstrate the role of data warehouses in improving operational visibility, performance measurement, and strategic planning in the shipping industry. This implementation uses SQL-based tools to create schemas, run queries, and provide a practical understanding of data storage in real-world logistics scenarios.

4. Methodology

The design and development of the "Data Warehouse for Tracking Delivery and Shipments" project involves several sequential stages, from data collection to reporting and visualization. The workflow follows the ETL → Warehouse → Features → Model → Deployment → Test → Report pipeline, providing a holistic approach to data management and analysis preparation. The following subsections describe each step-in detail.

4.1. Data Collection

The foundation of the project begins with the collection of relevant data that reflects actual shipping and delivery operations. For this purpose, Excel spreadsheets and CSV files are used as the main data sources, representing transaction records of various logistics activities such as product dispatch, delivery tracking, courier information, vehicle destination, customer details, etc. These files are either created manually or obtained from open-source repositories (e.g., simulated courier datasets) to represent:

- **Shipment Details** (shipment ID, date, origin, destination, distance, cost, delivery status, courier ID, customer ID, vehicle ID, route ID)
- **Courier Information** (courier ID, name, assigned region, performance rating)

- **Customer Details** (customer ID, name, location, type)
- **Vehicle Data** (vehicle ID, vehicle type, capacity)
- **Route Details** (route ID, source, destination, average time)

The data collected in these CSV/Excel files is saved in a staging area before being loaded to main storage. This preparation step is important for data cleaning and transformation.

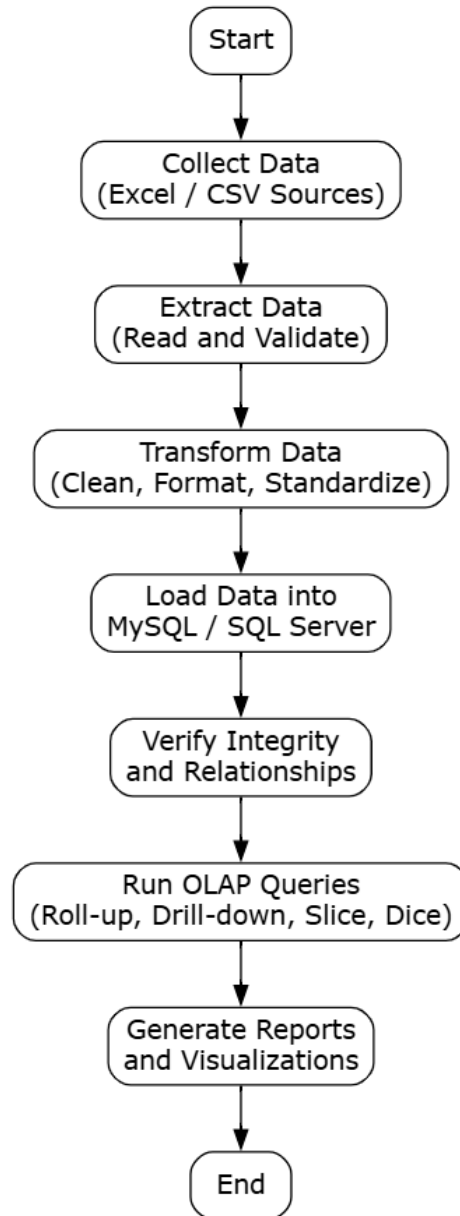


Figure 4.1. Flow Chart

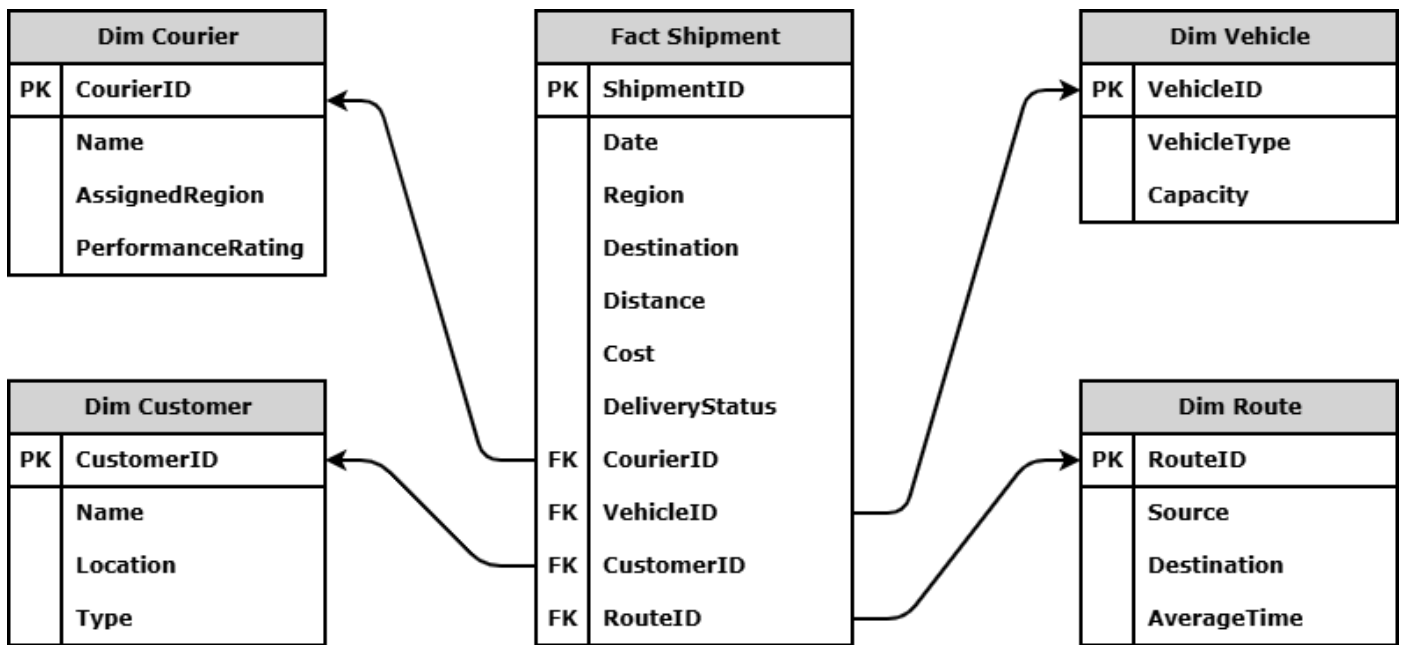
4.2. Data Warehousing

At the heart of the project is a data warehouse that consolidates all shipping-related information into one structured repository. This repository is implemented using MySQL/SQL Server, a robust relational database management system suitable for handling large amounts of analytical data. Both tools offer strong SQL support, built-in data import utilities, and efficient indexing mechanisms, making them ideal for classroom implementation and experimentation.

This storage follows a star architecture, which simplifies query execution and improves performance during OLAP (online analytical processing) operations. The schema includes a central fact table that stores quantitative data about shipments, such as cost, time, and delivery status, surrounded by several dimension tables that describe the contextual details of each shipment. This structure provides a clear and efficient way to organize data for analytical queries.

The star schema includes:

- **Fact Shipment Table:** Contains key performance metrics (measures) for each shipment, such as shipping time, cost, distance, and delivery status. This table also includes **Foreign Keys** that link it to all the dimension tables (CourierID, CustomerID, VehicleID, RouteID).
- **Dim Courier Table:** Stores courier details, acting as a dimension that describes the CourierID. Attributes include the courier's name, assigned region, and performance rating.
- **Dim Customer Table:** Stores customer information, acting as a dimension that describes the CustomerID. Attributes include the customer's name, region, and city.
- **Dim Vehicle Table:** Stores vehicle data, acting as a dimension that describes the VehicleID. Attributes include vehicle type, payload capacity, and registration number.
- **Dim Route Table:** Stores route definitions, acting as a dimension that describes the RouteID. Attributes include the route's origin, destination, and average travel time.



4.2. Project Star Schema (Fact and Dimensions Tables)

The framework enables complex analytical queries through roll-up, drill-down, slice, and split operations, allowing users to analyze shipping trends, assess courier service performance, and identify regional shipping patterns.

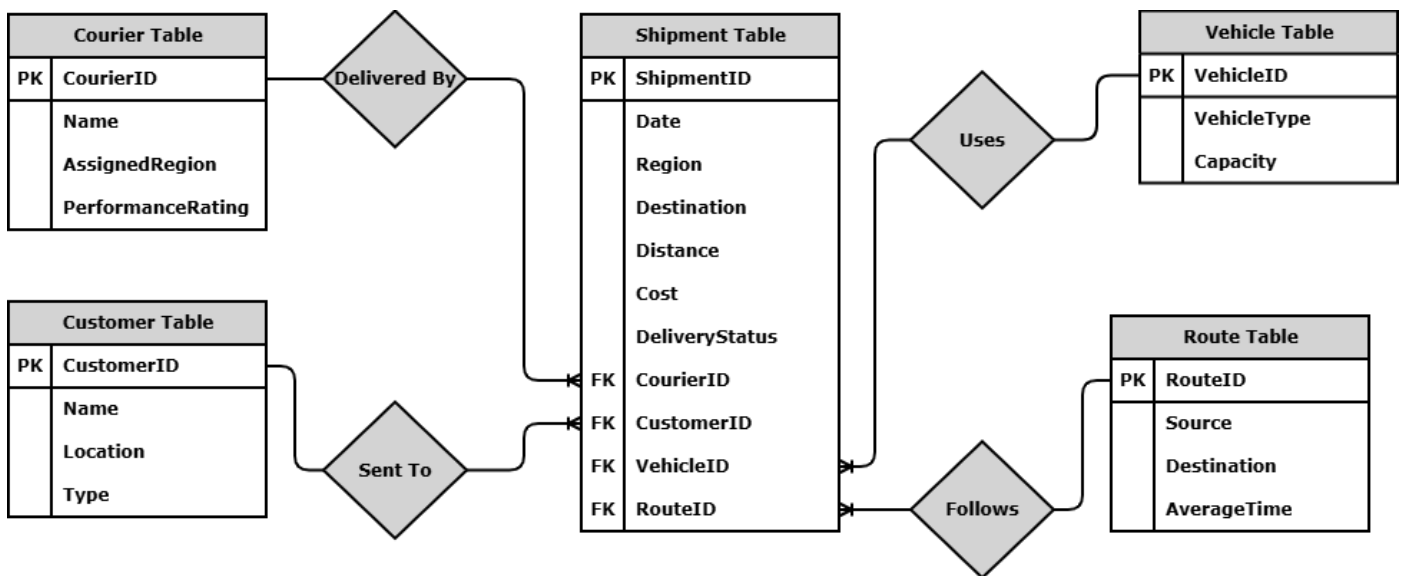


Figure 4.3. Entity Relationship Diagram (ERD)

4.3. Feature Engineering

Feature engineering, in the context of data warehousing, refers to the process of deriving new analytical attributes (features) from existing data to improve the quality and usability of the information stored in the

warehouse. After successfully implementing the ETL process and star schema, the project will focus on enriching the data by creating additional calculated columns and summary measures using SQL queries.

Several types of derived features are developed:

1. **Time-Based Features:** Using the date dimension generates new columns such as *Deliver_Duration* (difference between ship date and delivery date), *Day_of_Week*, and *Month_Name*. These features allow you to analyze time series, such as identifying peak shipping months and weekly shipping volumes.
2. **Performance Characteristics:** Functions such as *Average_delivery_Time_Per_Courier*, *OnTime_delivery_Rate*, and *Average_Cost_Per_Route* are calculated based on carrier and route data. These provide insight into the operational efficiency of couriers and delivery regions.
3. **Cost and Distance Metrics:** Based on the route size table and delivery facts, derived attributes such as *Cost_Per_Kilometer* and *Delivery_Speed_Index* (distance divided by time) are created to evaluate route profitability and optimization.
4. **Categorical Indicators:** Boolean or categorical metrics such as latency/activity time and high/medium/low distance are added to facilitate grouping and summarization in analytical queries.

These engineering functions are stored in the warehouse as additional columns or materialized views to support and improve reporting quality.

4.4. System Block Diagram

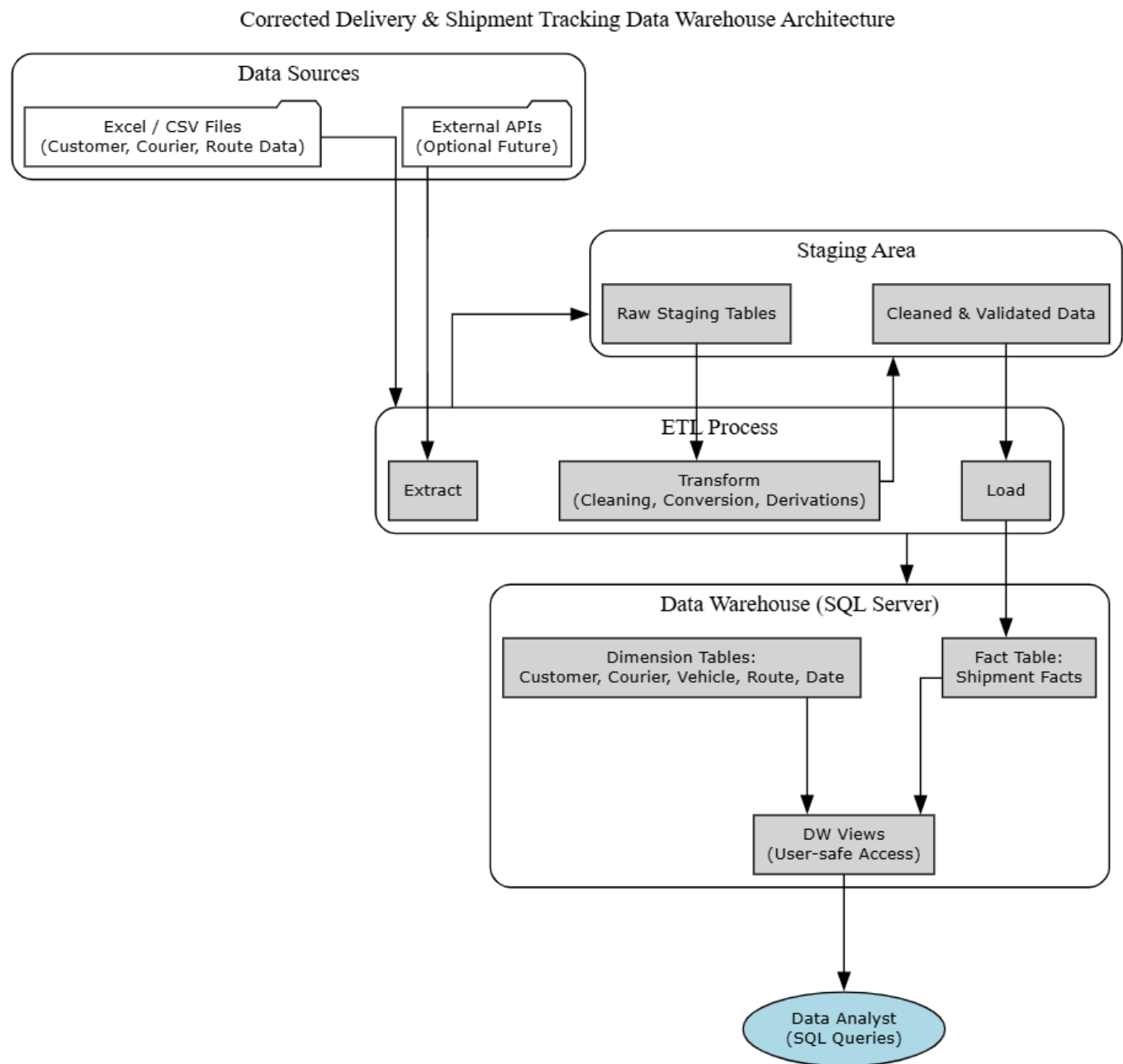


Figure 4.4. System Block Diagram

4.5. Model Building

In this project, the term "model building" refers to logical modeling of relationships between data in a warehouse, rather than predictive machine learning models. The star schema itself acts as an analytical model, providing efficient data organization and query performance.

The model includes:

- **Fact Table Relationships:** Each distributed record in the fact table references a dimension table through a primary key and foreign key relationship, ensuring referential integrity.
- **Aggregation Models:** SQL views are created to model key performance indicators (KPIs), such as:
 - Total Deliveries per Region
 - Average Delivery Cost per Courier
 - Monthly Delivery Volume Trends
- **Business Logic Modeling:** Triggers and stored procedures are implemented to automatically calculate updated metrics, such as recalculating average delivery time when new shipping data is loaded.

This relational model forms the basis for subsequent levels of analysis and reporting. This keeps your queries effective and results consistent across all reporting tools.

4.6. Deployment

Deployment is the process of making ready-made data stores and their associated queries available in a lab environment. The implementation runs on MySQL/SQL Server, so deployment can occur locally or over a shared lab network, allowing multiple students and faculty to access and run analytical queries.

The deployment steps include:

1. **Database Configuration:** The final storage schema (tables, relationships, indexes) is created in MySQL or SQL Server using the CREATE TABLE and ALTER TABLE statements.
2. **Load the Data:** Cleaned CSV or Excel data files are imported into warehouse tables using built-in database import tools.
3. **Stored Procedures and Views:** Frequently used queries (such as regional performance reports) are saved as SQL views or stored procedures for easy access.
4. **User Access:** Permissions are configured to allow instructors or other users to view reports, run queries, and make limited updates as needed.

This deployment makes the data warehouse a fully operational analytical environment, ready for testing and reporting.

4.7. Testing

Testing is an important step in ensuring that your data warehouse is correct, accurate, and efficient. The following tests are performed to verify design and data integrity:

1. **Data Validation Test:** Verify that all data loaded from Excel or CSV files matches the original source values. The number of records is compared before and after loading to ensure completeness.
2. **Schema Integrity Testing:** Ensures that all relationships between primary keys and foreign keys are correct. Referential integrity constraints are checked using SQL queries and database tools.
3. **ETL Accuracy Testing:** Verify that data transformations (such as date conversion, cost calculations, and delay classification) are applied correctly.

4. **Performance Testing:** Indexing and optimization techniques are evaluated to enable efficient execution of analytical queries even on large datasets.

Testing ensures that the data warehouse is logical and can provide consistent analytical results across all queries and reports.

4.8. Reporting

The final step in the project is to create an analytical report that summarizes the information obtained from the data warehouse. Reports are run using SQL-based queries and Excel visualization tools. The objective is to demonstrate how data warehouses can support business decision making through data analysis.

Key reports include:

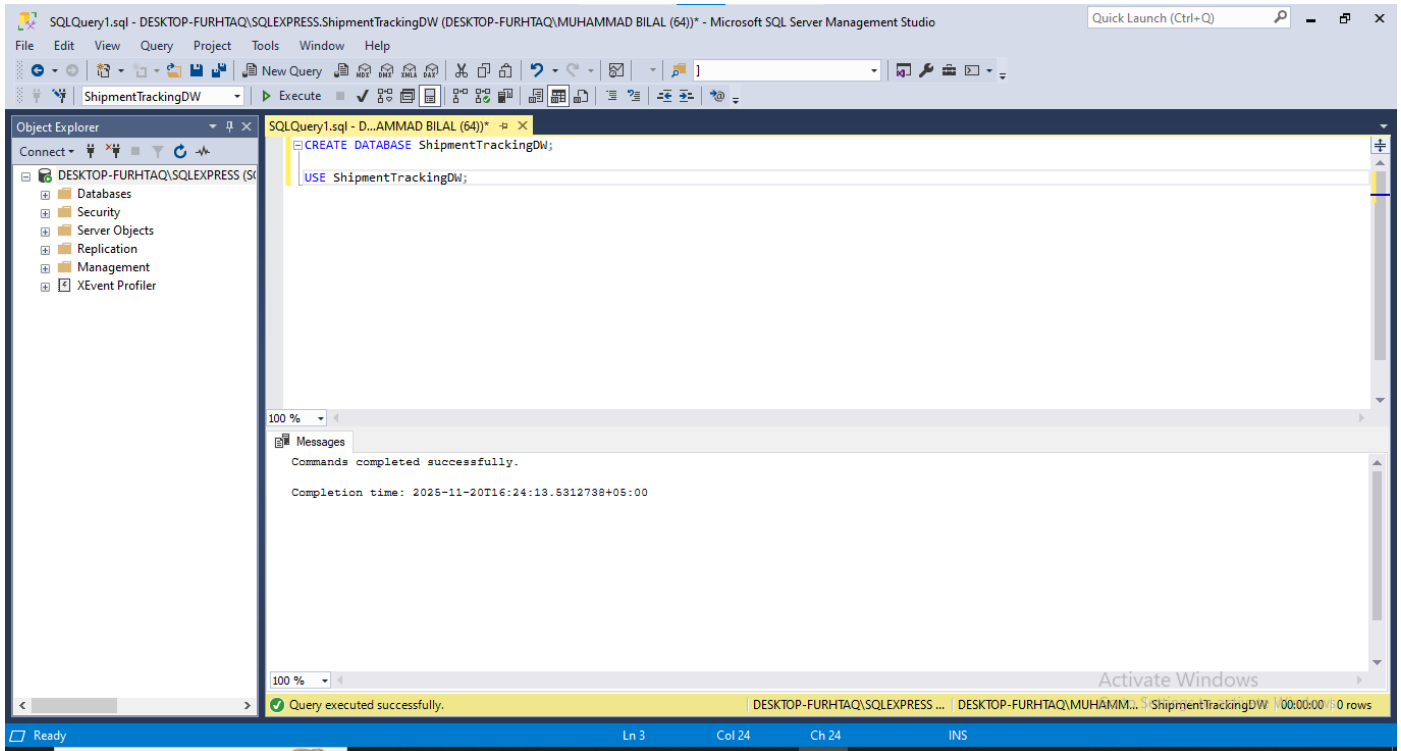
1. **Shipment Summary Report:** Shows total shipments, total cost, and average delivery duration by region or courier.
2. **Courier Performance Report:** Compares couriers based on on-time delivery percentage and average shipment cost.
3. **Route Analysis Report:** Displays the most frequently used routes and their efficiency metrics.
4. **Monthly Trend Report:** Visualizes shipment trends over months or quarters to identify seasonal variations.

Reports are generated through SQL SELECT, GROUP BY, and JOIN queries and then exported to Excel for chart generation (bar charts, pie charts, and line graphs).

These reports help explain how a well-designed data warehouse can improve operational visibility and provide actionable insights to support decision-making in logistics and delivery management.

5. Results

Create and Use Database:



DimCourier Table:

```
CREATE TABLE DimCourier (  
    CourierID INT PRIMARY KEY,  
    CourierName NVARCHAR(100) NOT NULL,  
    AssignedRegion NVARCHAR(100),  
    PerformanceRating DECIMAL(3,2) CHECK (PerformanceRating BETWEEN 0 AND 5)  
);
```

DimCustomer Table:

```
CREATE TABLE DimCustomer (  
    CustomerID INT PRIMARY KEY,  
    CustomerName VARCHAR(100) NOT NULL,  
    Location VARCHAR(200),  
    CustomerType VARCHAR(50)  
);
```

DimVehicle Table:

```
CREATE TABLE DimVehicle (  
    VehicleID INT PRIMARY KEY,  
    VehicleType NVARCHAR(50) NOT NULL,  
    Capacity INT CHECK (Capacity >= 0)  
);
```

DimRoute Table:

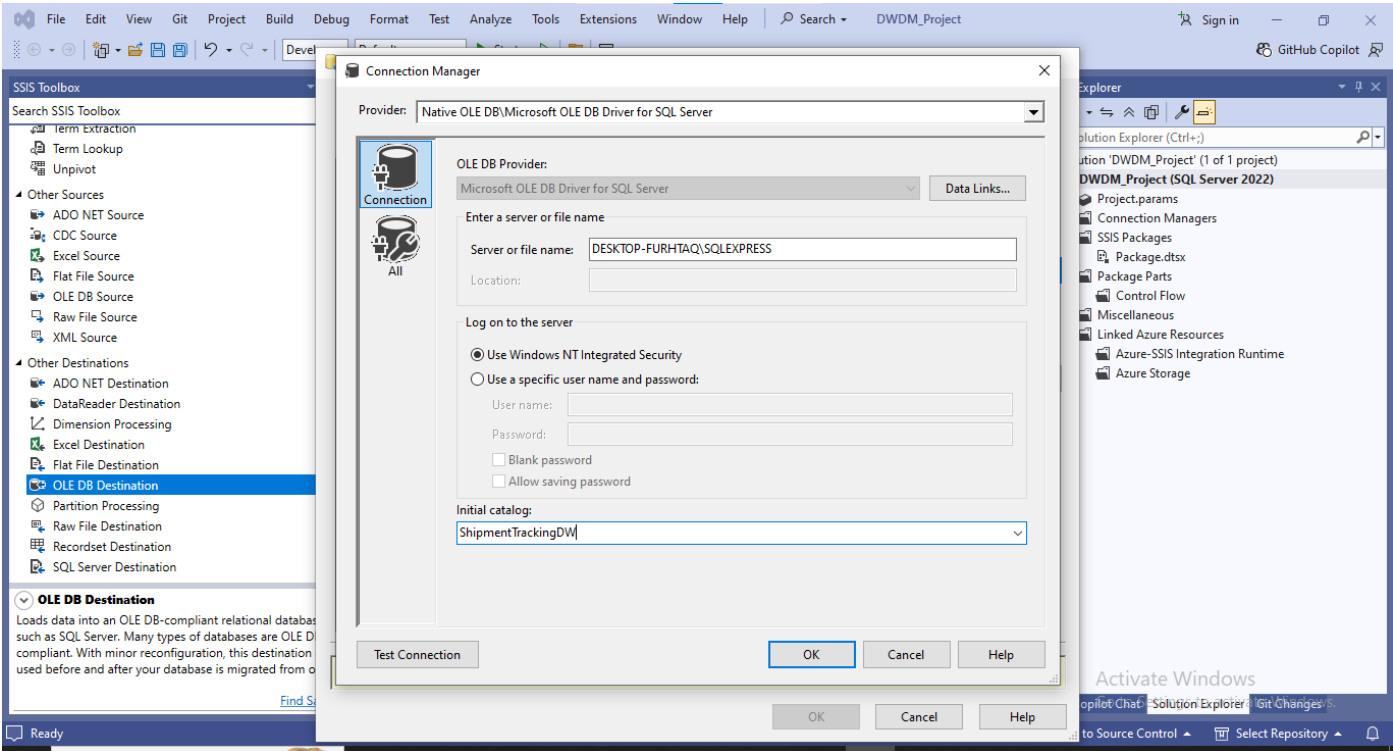
```
CREATE TABLE DimRoute (  
    RouteID INT PRIMARY KEY,  
    Origin NVARCHAR(100),  
    Destination NVARCHAR(100),  
    DistanceKM DECIMAL(10,2) CHECK (DistanceKM >= 0)  
);
```

FactShipment Table:

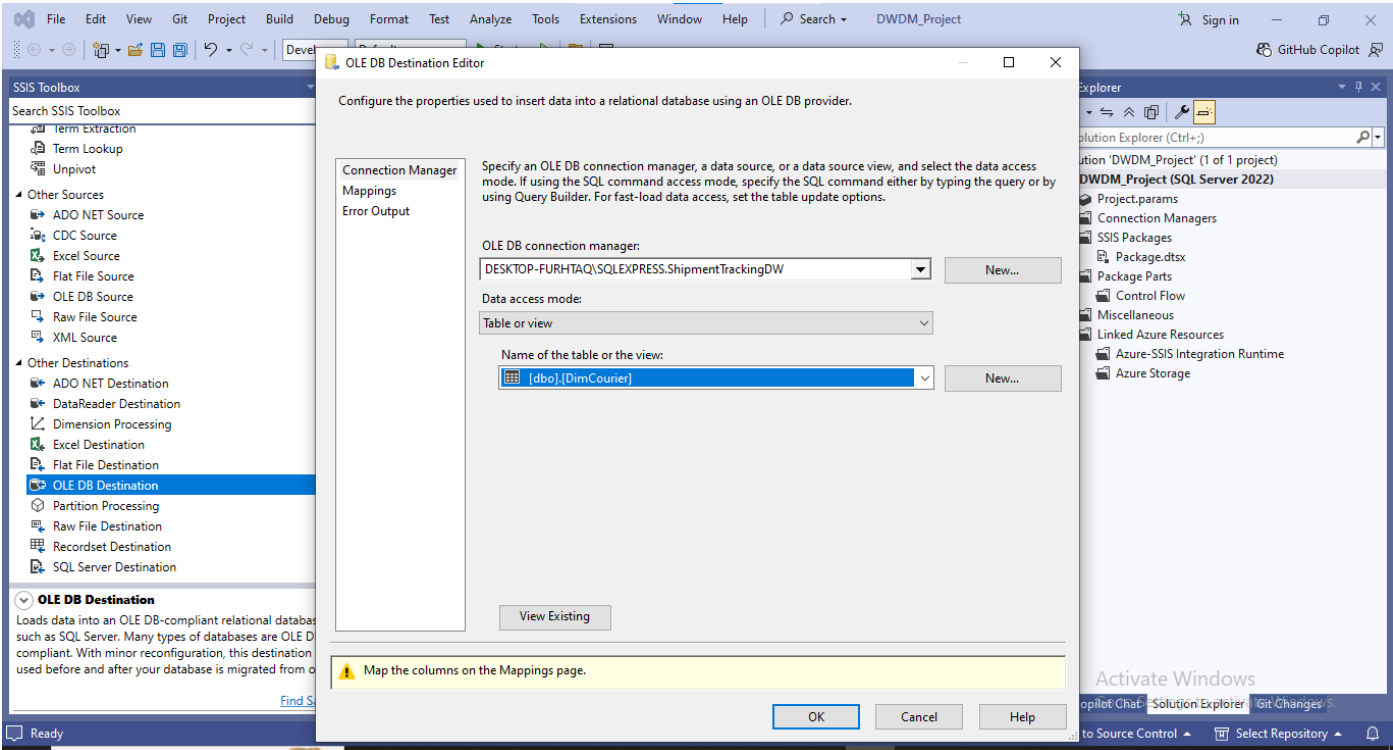
```
CREATE TABLE FactShipment (  
    ShipmentID INT PRIMARY KEY,  
    ShipmentDate DATE NOT NULL,  
    Origin NVARCHAR(100),  
    Destination NVARCHAR(100),  
    DistanceKM DECIMAL(10,2) CHECK (DistanceKM >= 0),  
    Cost DECIMAL(12,2) CHECK (Cost >= 0),  
    DeliveryStatus NVARCHAR(50),  
    CourierID INT,  
    CustomerID INT,  
    VehicleID INT,  
    RouteID INT,  
  
    CONSTRAINT FK_Shipment_Courier  
        FOREIGN KEY (CourierID) REFERENCES DimCourier(CourierID),  
  
    CONSTRAINT FK_Shipment_Customer  
        FOREIGN KEY (CustomerID) REFERENCES DimCustomer(CustomerID),  
  
    CONSTRAINT FK_Shipment_Vehicle  
        FOREIGN KEY (VehicleID) REFERENCES DimVehicle(VehicleID),  
  
    CONSTRAINT FK_Shipment_Route  
        FOREIGN KEY (RouteID) REFERENCES DimRoute(RouteID)  
);
```

Inserting Data into Tables Using ELT:

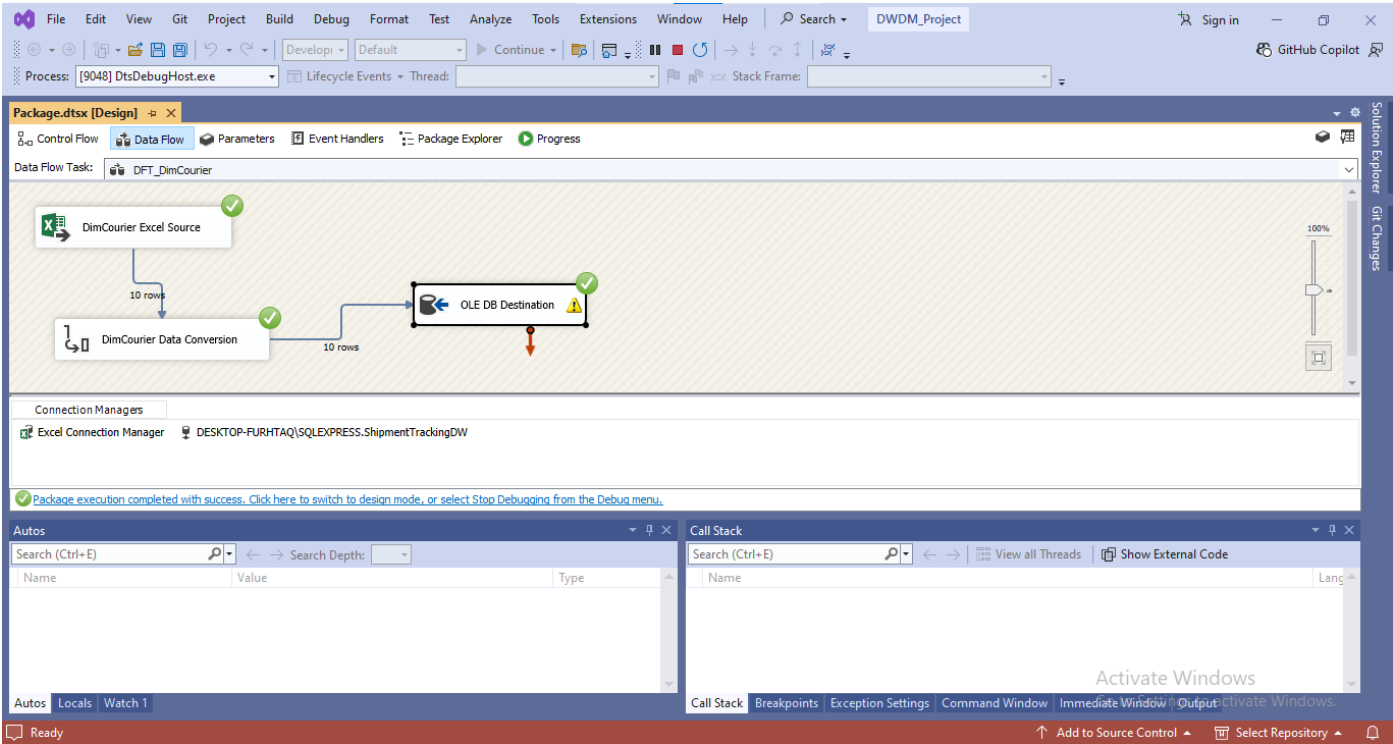
Configure OLE DB Destination (screen shot of final steps)



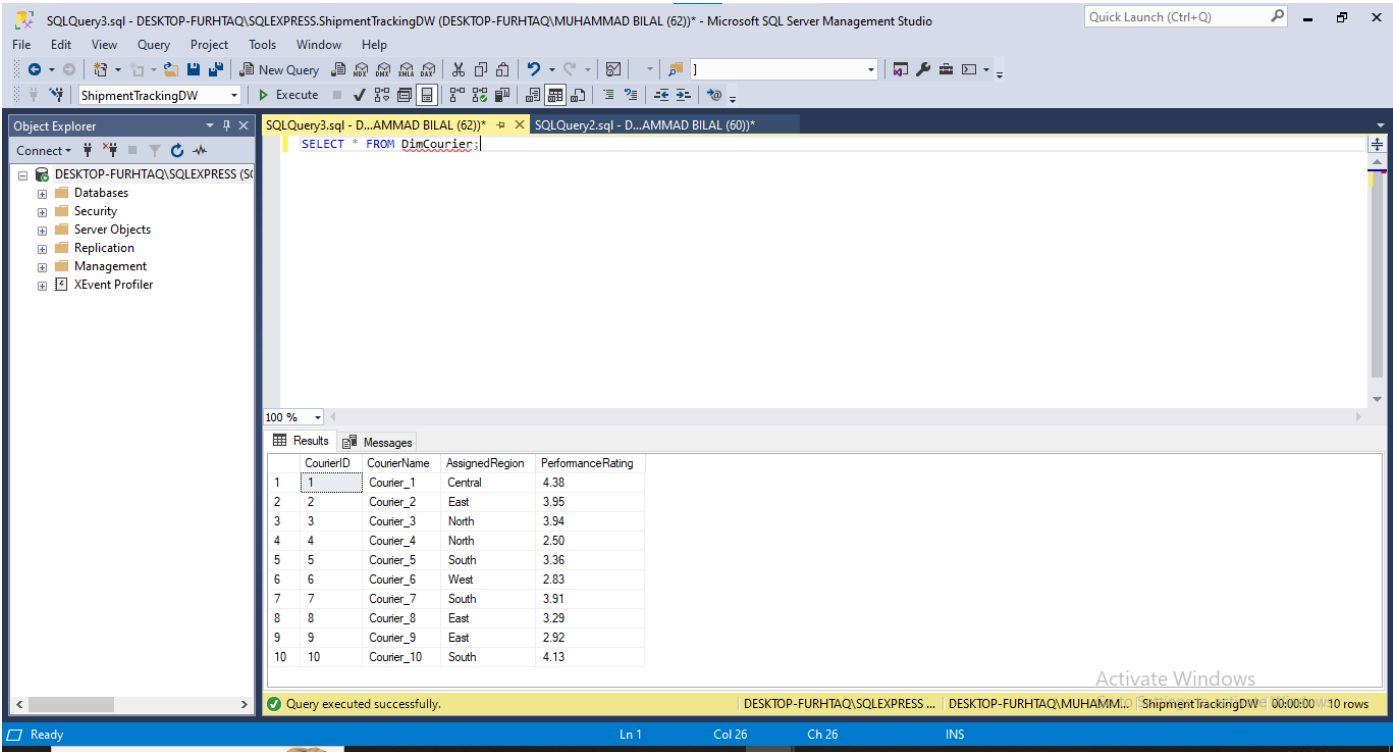
Select Server Table



Run the DFT

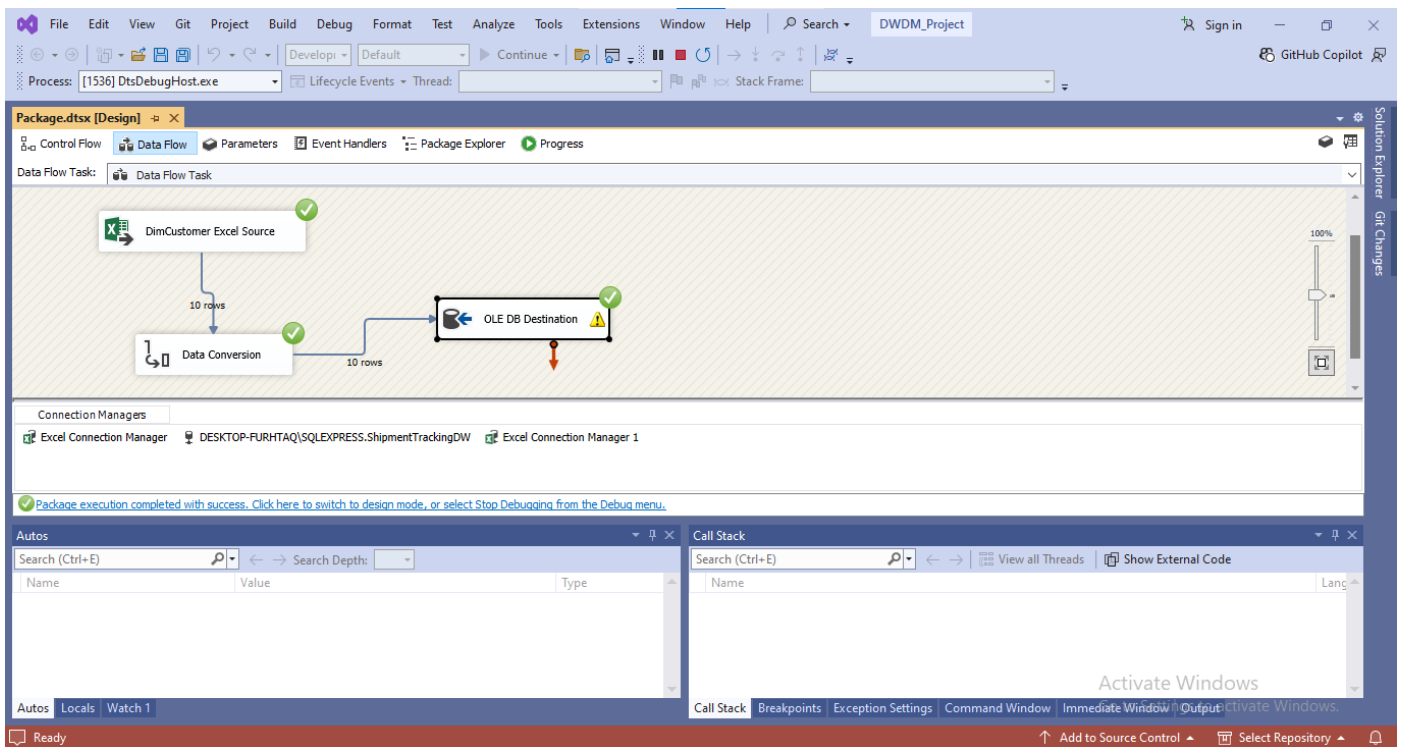


Conformation through SSMS

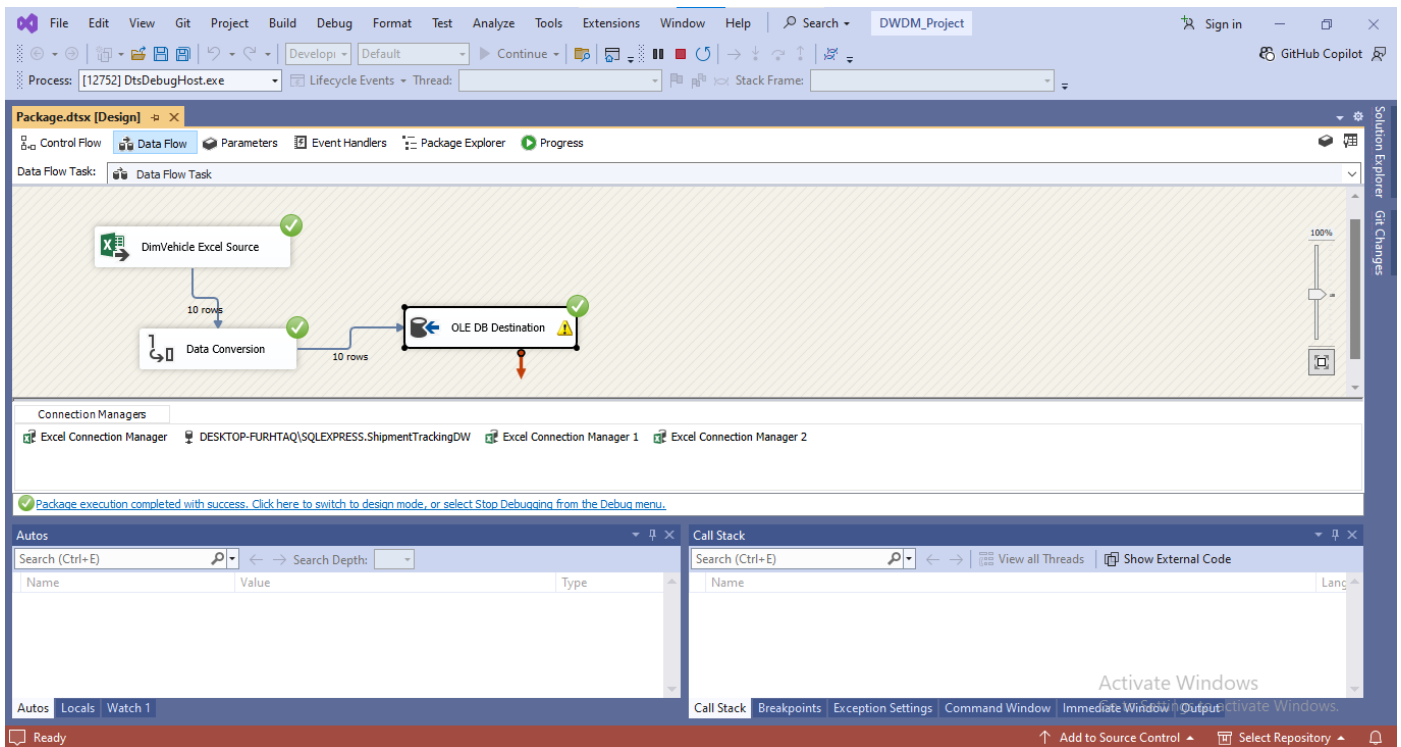


Create DFT for rest of Tables

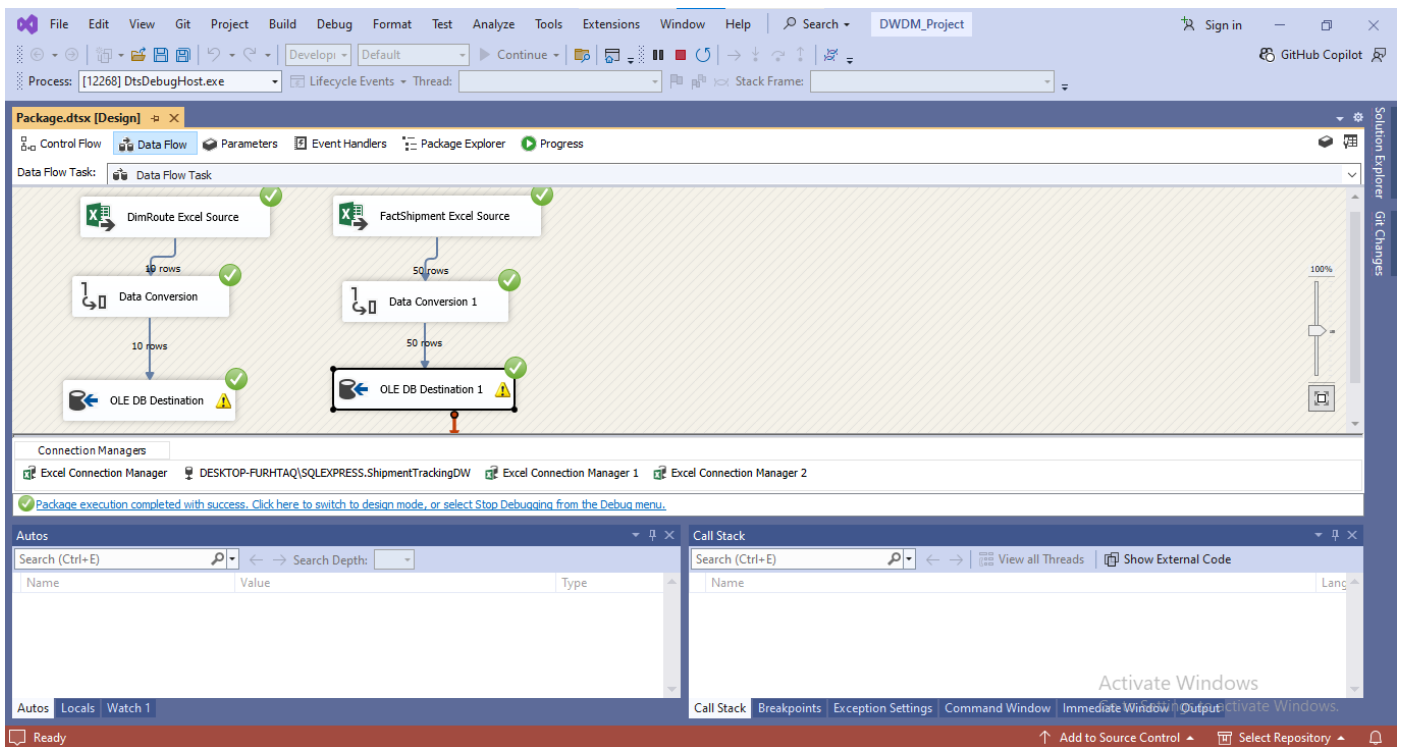
DimCustomer



DimVehicle



DimRoute and FactShipment



Features of Project

```
-- Day_of_Week
SELECT
    ShipmentID,
    ShipmentDate,
    DATENAME(WEEKDAY, ShipmentDate) AS Day_of_Week
FROM FactShipment;
```

100 %

	ShipmentID	ShipmentDate	Day_of_Week
1	1	2024-04-22	Monday
2	2	2024-10-21	Monday
3	3	2024-11-29	Friday
4	4	2024-07-14	Sunday
5	5	2024-03-26	Tuesday
6	6	2024-07-09	Tuesday
7	7	2024-07-09	Tuesday
8	8	2024-04-15	Monday
9	9	2024-12-20	Friday
10	10	2024-05-02	Thursday
11	11	2024-04-02	Tuesday

✓ Query executed successfully. | DESKTOP-FURHTAQ\SQLEXPRESS ...

```
-- Month_Name
SELECT
    ShipmentID,
    ShipmentDate,
    DATENAME(MONTH, ShipmentDate) AS Month_Name
FROM FactShipment;
```

100 %

	ShipmentID	ShipmentDate	Month_Name
1	1	2024-04-22	April
2	2	2024-10-21	October
3	3	2024-11-29	November
4	4	2024-07-14	July
5	5	2024-03-26	March
6	6	2024-07-09	July
7	7	2024-07-09	July
8	8	2024-04-15	April
9	9	2024-12-20	December
10	10	2024-05-02	May
11	11	2024-04-02	April

Query executed successfully. DESKTOP-FURH

```
-- Average Shipments Per Courier
SELECT
    C.CourierName,
    COUNT(F.ShipmentID) AS Total_Shipments
FROM FactShipment F
JOIN DimCourier C
    ON F.CourierID = C.CourierID
GROUP BY C.CourierName;
```

100 %

	CourierName	Total_Shipments
1	Courier_1	4
2	Courier_10	5
3	Courier_2	3
4	Courier_3	5
5	Courier_4	5
6	Courier_5	8
7	Courier_6	5
8	Courier_7	2
9	Courier_8	7
10	Courier_9	6

Query executed successfully.

```
-- Average Cost Per Courier
SELECT
    C.CourierName,
    AVG(F.Cost) AS Average_Cost
FROM FactShipment F
JOIN DimCourier C
    ON F.CourierID = C.CourierID
GROUP BY C.CourierName;
```

100 %

Results Messages

	CourierName	Average_Cost
1	Courier_1	1115.070000
2	Courier_10	1426.720000
3	Courier_2	1227.246666
4	Courier_3	1237.552000
5	Courier_4	713.748000
6	Courier_5	935.130000
7	Courier_6	1452.462000
8	Courier_7	1480.245000
9	Courier_8	1000.355714
10	Courier_9	1326.211666

Query executed successfully.

```
-- Average Cost Per Route
SELECT
    C.CourierName,
    AVG(F.Cost) AS Average_Cost
FROM FactShipment F
JOIN DimCourier C
    ON F.CourierID = C.CourierID
GROUP BY C.CourierName;
```

100 %

Results Messages

	CourierName	Average_Cost
1	Courier_1	1115.070000
2	Courier_10	1426.720000
3	Courier_2	1227.246666
4	Courier_3	1237.552000
5	Courier_4	713.748000
6	Courier_5	935.130000
7	Courier_6	1452.462000
8	Courier_7	1480.245000
9	Courier_8	1000.355714
10	Courier_9	1326.211666

Query executed successfully.

```
-- Cost Per Kilometer
SELECT
    ShipmentID,
    Cost,
    DistanceKM,
    Cost / NULLIF(DistanceKM, 0) AS Cost_Per_KM
FROM FactShipment;
```

100 %

Results Messages

	ShipmentID	Cost	DistanceKM	Cost_Per_KM
1	1	1346.11	1422.26	0.9464584534473
2	2	1192.89	825.11	1.4457345081261
3	3	326.58	311.84	1.0472678296562
4	4	1517.32	1422.26	1.0668372871345
5	5	1863.89	1162.92	1.6027671722904
6	6	609.01	311.84	1.9529566444330
7	7	1434.86	825.11	1.7389923767740
8	8	1893.09	1360.33	1.3916402637595
9	9	1871.93	1162.92	1.6096808034946
10	10	286.44	296.12	0.9673105497771
11	11	518.46	296.12	1.7508442523301

Query executed successfully.

6. Conclusion

As part of the “Data warehouse for tracking deliveries and shipments” project, a reliable analytics platform for the logistics and shipping sector was developed, implemented and successfully tested. Adhering to data warehouse fundamentals, a star schema was created to efficiently integrate disparate operational data (from CSV/Excel sources) related to deliveries, couriers, customers, vehicles, and routes.

The ETL process demonstrated the ability to clean, transform, and load this disparate data into a single structure in a relational database environment (MySQL/SQL Server). Powered by feature engineering to generate derived metrics such as *Delivery_Duration* and *OnTime_delivery_Rate*, this unified repository provides a solid foundation for in-depth analysis.

In conclusion, this academic prototype demonstrates how a well-structured data warehouse can transform raw logistics data into actionable insights, supporting operational efficiency and strategic planning by enabling decision makers to effectively track key performance indicators (KPIs), identify delivery bottlenecks, benchmark delivery service performance, and predict future supply trends.

7. References

[1] R. Kimball and M. Ross, *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*, 3rd ed. Hoboken, NJ, USA: Wiley, 2013.

- [2] W. H. Inmon, *Building the Data Warehouse*, 4th ed. Indianapolis, IN, USA: Wiley, 2005.
- [3] MySQL, “MySQL 8.0 Reference Manual,” *MySQL Documentation*. Available: <https://dev.mysql.com/doc/refman/8.0/en/>.
- [4] Microsoft, “SQL Server documentation,” *Microsoft Learn*. Available: <https://learn.microsoft.com/en-us/sql/sql-server/>.